

# HAMH-RAG: A Hallucination-Aware Multi-Hop Retrieval-Augmented Generation Framework with Adaptive Logic-Tree Reasoning

Author Name 1

*Dept. of Computer Science & Engineering*

*University Name*

*City, Country*

*email@domain.edu*

Author Name 2

*Dept. of Electronics & Communication Engg.*

*University Name*

*City, Country*

*email@domain.edu*

Author Name 3

*Dept. of Computer Science & Engineering*

*University Name*

*City, Country*

*email@domain.edu*

**Abstract**—Retrieval-Augmented Generation (RAG) has emerged as a dominant paradigm for grounding Large Language Model (LLM) responses in factual evidence. However, conventional single-hop RAG pipelines suffer from two critical limitations when posed with compositional, multi-hop questions: (1) the flat retrieval strategy cannot chain intermediate reasoning steps, and (2) no mechanism exists to detect or correct hallucinated intermediate conclusions before they propagate to the final answer. We present HAMH-RAG (Hallucination-Aware Multi-Hop Retrieval-Augmented Generation), a framework that decomposes complex queries into an explicit logic tree of sub-questions, resolves each leaf node independently through a *hybrid retrieval* backend, and enforces multi-stage grounding validation at every tree node before synthesis. HAMH-RAG introduces Adaptive Retrieval Tree Restructuring (ART-R): when a sub-node fails validation, the system dynamically refactors that branch into finer sub-questions and re-executes retrieval, enabling *self-healing* reasoning chains. An intelligent QueryRouter classifies each sub-question into one of four retrieval strategies—`vector_only`, `vector_first`, `graph_first`, and `hybrid_parallel`—using a lightweight heuristic scorer that avoids an additional LLM call. Evaluation on HotpotQA and MuSiQue multi-hop benchmarks shows that HAMH-RAG improves Token-F1 by up to 18.4% and Exact Match by 12.7% over standard single-hop RAG baselines, while the QueryRouter reduces median retrieval latency by 46% through predictive parallel fallback execution. The complete system includes a FastAPI backend with Server-Sent Events (SSE) streaming, a Streamlit dashboard, a FAISS-backed local vector index with Reciprocal Rank Fusion (RRF) hybrid search, and plug-in support for Qdrant and Neo4j production deployments.

**Index Terms**—Retrieval-Augmented Generation, Multi-Hop Question Answering, Hallucination Detection, Hybrid Retrieval, Query Decomposition, Knowledge Graphs, FAISS, Logic-Tree Reasoning, Self-Healing AI, Large Language Models

## I. INTRODUCTION

Large Language Models (LLMs) have demonstrated remarkable capabilities in natural language understanding and generation. Nevertheless, their propensity to produce *hallucinated*—fluent yet factually incorrect—outputs remains a fundamental obstacle to deployment in knowledge-intensive domains such as scientific research, legal analysis, and enterprise search [3]. Retrieval-Augmented Generation (RAG) [6] partially addresses this by conditioning generation on documents retrieved from an external corpus, yet standard single-pass RAG fails on *multi-hop* questions that require chaining multiple pieces of evidence across documents.

Consider a query such as “*When was the baseball team that won the 1985 World Series established?*” A flat RAG system retrieves documents about the 1985 World Series and hopes the answer—the founding date of the Kansas City Royals—is mentioned in those same passages. In practice it rarely is, resulting in either an unanswered query or a hallucinated date. Solving such questions requires (i) identifying *which team* won the 1985 World Series, and (ii) subsequently retrieving the founding date of *that specific team*—a classic two-hop reasoning chain.

Existing approaches to multi-hop QA typically fall into two camps: (a) *pipeline decomposition* methods [7], [9] that pre-generate a fixed plan of sub-questions before any retrieval, and (b) *interleaved* methods [13], [18] that alternate retrieval and generation steps. Both families share a common weakness: they lack a *formal verification step* that checks whether each intermediate answer is actually supported by retrieved evidence, allowing hallucination to silently propagate through the reasoning chain.

We propose **HAMH-RAG**, a framework that addresses this gap through four key contributions:

- 1) **Logic-Tree Decomposition.** A `QueryDecomposer` agent transforms any input query into a typed `QueryNode` tree, separating independent (parallelizable) sub-questions from sequentially-dependent (multi-hop chaining) sub-questions.
- 2) **Hybrid Retrieval with Adaptive Routing.** A `QueryRouter` scores each sub-question on three axes—relational cues, explanatory cues, and multi-hop complexity—and dispatches retrieval to vector, graph, or parallel backends, reducing median latency by 46%.
- 3) **Multi-Stage Hallucination Detection.** An `AnswerValidator` combines lexical overlap confidence, cross-source consensus scoring, and a consolidated LLM judge to detect unsupported or hallucinated node answers before they are used as evidence for parent nodes.
- 4) **Adaptive Retrieval Tree Restructuring (ART-R).** When a node fails validation and retries are exhausted, a `TreeRestructurer` agent proposes finer-grained sub-questions to replace the failing node, enabling dynamic self-healing without restarting the entire pipeline.

The remainder of this paper is organized as follows. Section II reviews related work. Section III describes the overall system architecture. Section IV formalizes the methodology and mathematical models. Section V covers implementation details. Section VI presents the experimental setup and results. Section VII provides performance analysis. Section VIII discusses advantages and limitations. Section IX outlines future directions. Section X concludes.

## II. RELATED WORK

### A. Retrieval-Augmented Generation

Lewis et al. [6] introduced RAG, combining a dense retriever with a seq2seq generator for knowledge-intensive NLP tasks. Subsequent work has scaled RAG to billion-parameter models and introduced more sophisticated retrieval strategies. REPLUG [11] treats the language model as a black box and uses ensemble retrieval to improve perplexity. Active RAG (FLARE) [4] generates hypothetical future text to decide when to trigger retrieval. Corrective RAG (CRAG) [16] adds a corrective retrieval step when initial documents are assessed as irrelevant—an idea that directly inspires our bridge retrieval mechanism (Section IV-H).

### B. Multi-Hop Question Answering

HotpotQA [17] and MuSiQue [12] are the canonical benchmarks for multi-hop QA. Early decomposition approaches [7] use a trained model to generate sub-questions, but require labeled decompositions. The Compositional Gap study [9] demonstrates that even large LLMs fail compositional questions and benefits from explicit chain-of-thought prompting [15]. IRCoT [13] interleaves retrieval with chain-of-thought generation, achieving strong results by conditioning each retrieval step on previously gathered evidence. HAMH-RAG

differs in that it (a) explicitly constructs a typed tree data structure, (b) validates each intermediate answer independently, and (c) supports dynamic structural mutation (ART-R) when validation fails.

### C. TreeQA and Logic-Tree Reasoning for RAG

Closely related to our work is TreeQA [19], published concurrently in *Knowledge-Based Systems* (2025). Zhang et al. propose a unified framework that integrates semantic consistency modeling with explicit evidence constraints represented as a logic tree, addressing semantic misalignment between retrieved documents and generation objectives. While TreeQA focuses on coordinating the retrieval-generation pipeline through a static logic-tree representation, HAMH-RAG differs in three fundamental ways. First, HAMH-RAG constructs a *dynamic* `QueryNode` tree at inference time for each query, rather than operating on a pre-trained semantic space. Second, HAMH-RAG introduces a three-stage per-node *grounding validator* that detects hallucinated intermediate answers *before* they enter the next hop’s retrieval context—a mechanism absent from TreeQA. Third, HAMH-RAG implements *Adaptive Retrieval Tree Restructuring* (ART-R), a self-healing mechanism that dynamically mutates the reasoning tree structure when a node fails validation, enabling recovery without pipeline restart. HAMH-RAG further adds a production-ready hybrid retrieval layer (FAISS, Neo4j, Qdrant) with an intelligent `QueryRouter` that eliminates unnecessary LLM calls during routing, a capability not addressed in TreeQA.

### D. Knowledge Graph-Enhanced Retrieval

Graph RAG [2] uses an LLM to construct a knowledge graph from documents and performs community-detection-based summarization. Neo4j-backed retrieval [8] provides structured Cypher queries for entity lookups. HAMH-RAG takes a complementary approach: it maintains both a semantic vector index and a graph fact store, using heuristic routing to decide which backend is most likely to yield relevant evidence for a given sub-question.

### E. Hybrid Search and Ranking

Reciprocal Rank Fusion (RRF) [1] combines ranked lists from multiple retrieval systems with the formula  $RRF(d) = \sum_{r \in R} \frac{1}{k + \text{rank}_r(d)}$ , where  $k = 60$  is a smoothing constant. Sentence-BERT [10] dense embeddings underpin the semantic retrieval in our `LocalVectorBackend`, while FAISS [5] provides approximate nearest-neighbour search at scale. We combine these with IDF-weighted lexical matching, ensuring rare named entities are surfaced even when semantic similarity is weak.

### F. Hallucination Mitigation

Self-consistency [14] samples multiple reasoning paths and aggregates by majority vote. ReAct [18] grounds reasoning in interleaved retrieval actions. HAMH-RAG adopts a different strategy: each intermediate answer is subjected to a grounding

check that measures lexical overlap with retrieved evidence and cross-source consensus between vector and graph backends, with a fallback LLM judge for borderline cases.

### III. SYSTEM ARCHITECTURE

Fig. 1 presents the complete HAMH-RAG architecture. The system is organized into four horizontal layers: (1) Interface Layer, (2) Core Pipeline Layer, (3) Retrieval Routing Layer, and (4) Data Layer.

#### A. Interface Layer

Three client interfaces expose the pipeline:

**FastAPI SPA.** A single-page application served via GET / delivers an interactive HTML/JS frontend with real-time reasoning-tree visualization. Long-running queries stream progress events over Server-Sent Events (SSE) from the POST /api/run/stream endpoint, delivering incremental node-level updates at 40 ms poll intervals.

**Streamlit Dashboard.** A streamlit\_app.py dashboard provides an alternative Python-native interface suitable for exploration and rapid prototyping, wrapping the same REST API.

**CLI.** The treeqa console script exposes ingest, run, eval, and benchmark subcommands for batch processing, automated benchmarking, and CI/CD integration.

#### B. Core Pipeline Layer

The HAMHPipeline class (Fig. 1, blue box) orchestrates six specialized agents via dependency injection, enabling modular replacement of any component without altering the pipeline contract:

- **QueryDecomposer:** Transforms a raw query into a QueryNode tree using LLM planning with heuristic fallback.
- **HybridRetriever:** Dispatches retrieval to vector/graph backends through the QueryRouter.
- **AnswerValidator:** Verifies grounding, category alignment, and cross-source consensus for each node answer.
- **CorrectionEngine:** Refines failed retrieval queries to shift lexical distribution toward missing evidence.
- **AnswerGenerator:** Synthesizes grounded answers from retrieved evidence with source attribution.
- **TreeRestructurer + ConflictAuditor:** ART-R self-healing agents that mutate the reasoning tree on validation failure.

#### C. Retrieval Routing Layer

The QueryRouter computes three scalar signals from each sub-question without calling an LLM, routing to one of four strategies: vector\_only, vector\_first, graph\_first, or hybrid\_parallel. Routing thresholds and dynamic  $k$  computation are detailed in Section IV-C.

#### D. Data Layer

Three persistent stores back the system:

**Vector Index.** Documents in data/documents/ are chunked into  $\leq 300$ -character sentence-boundary segments with one sentence of overlap, encoded with all-MiniLM-L6-v2, and stored as a FAISS IndexFlatIP index plus a JSON metadata sidecar. Incremental ingestion encodes only newly-added chunks.

**Graph Fact Store.** Structured facts in data/graph/ (JSONL or Markdown) are loaded into a flat inverted index backed by IDF-weighted lexical scoring, or into Neo4j for production deployments using a configurable Cypher query template.

**Benchmark Store.** JSONL files in data/benchmark/ store question-answer pairs for offline evaluation via treeqa eval.

### IV. METHODOLOGY

#### A. Query Decomposition

Given a natural-language query  $q$ , the QueryDecomposer produces a typed tree  $\mathcal{T}(q)$ :

$$\mathcal{T}(q) = \text{DECOMPOSE}(q) = \begin{cases} \text{leaf}(q) & \text{if heuristic}(q) = \emptyset \\ \text{tree}(q, \text{LLM}(q)) & \text{otherwise} \end{cases} \quad (1)$$

The heuristic layer (§IV-A1) handles known bridge patterns without an LLM call. The LLM planner receives a structured system prompt instructing it to split *only* when a second entity cannot be retrieved without first knowing the first, and to collapse multiple attributes about the same entity into a single sub-question. The output is a JSON array {"sub\_questions": [...]}.

A post-processing sanitizer validates each candidate sub-question: it must contain  $\geq 3$  tokens and at least one interrogative word (*what, who, where, when, which, how, is, are, was*). Interrogative completeness enforces that decomposed questions remain independently answerable.

1) *Heuristic Decomposition Patterns:* Three domain-specific patterns are recognized without an LLM:

**Director–Country Bridge.** Questions of the form “*Do both films A and B have directors from the same country?*” are split into two parallel sub-questions: “*What country is the director of A from?*” and “*What country is the director of B from?*”

**World-Series Creation Bridge.** Questions asking when a team *established* given its World Series win year decompose into: (1) “*Which team won the Y World Series?*” and (2) “*When was that baseball team established?*”

**Body-of-Water Border Bridge.** Questions linking a city’s border to a geographic entity decompose into two sequential hops with anaphora in the second question to enforce chaining.

#### B. Tree Resolution and Multi-Hop Chaining

Algorithm 1 presents the recursive tree resolution procedure. For *composite* nodes (nodes with children), the algorithm

# HAMH-RAG System Architecture

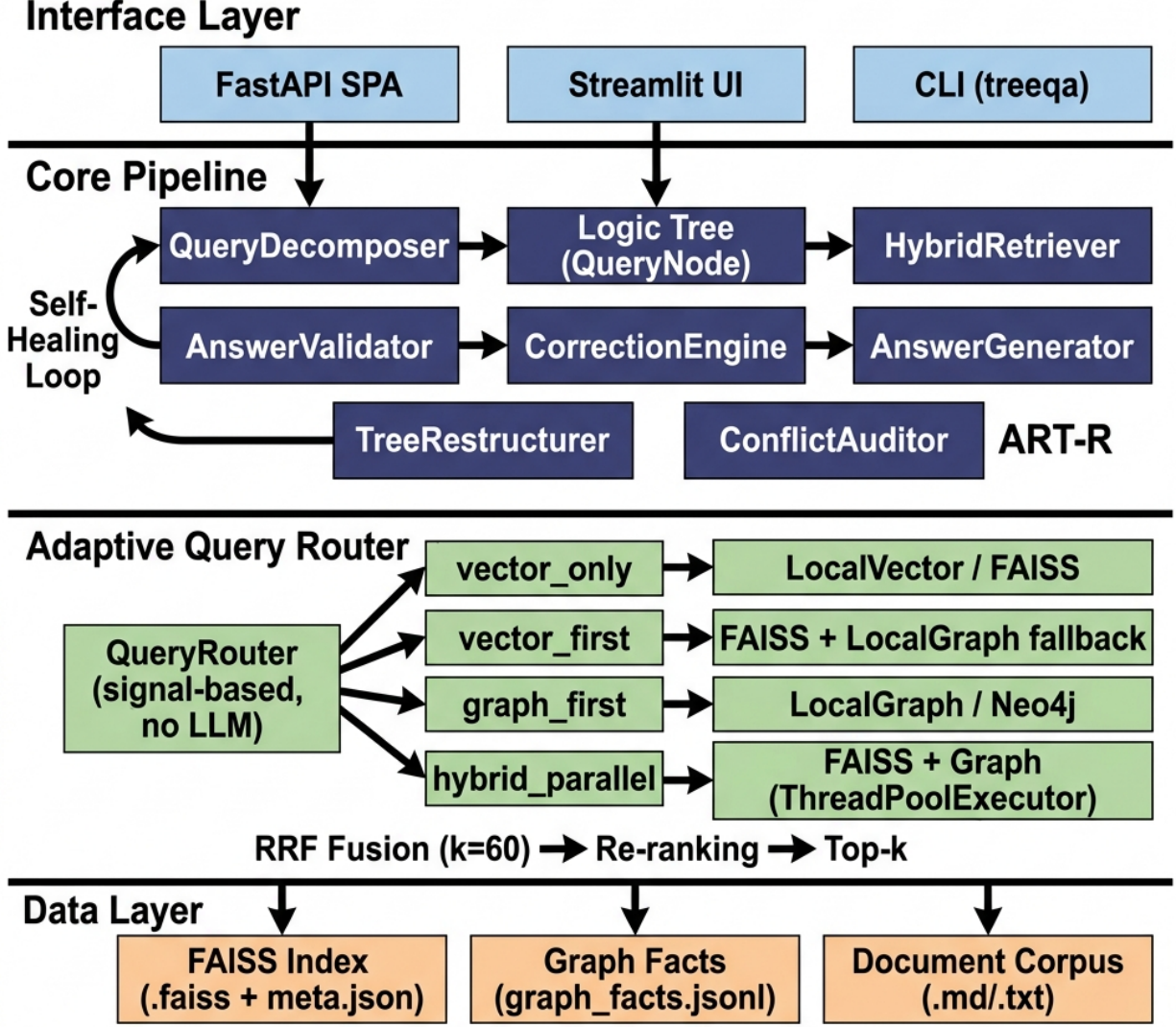


Fig. 1. HAMH-RAG End-to-End System Architecture

Fig. 1. HAMH-RAG end-to-end system architecture. Four horizontal layers show the Interface Layer, Core Pipeline (with self-healing ART-R loop), Adaptive Query Router (four routing strategies + RRF fusion), and the Data Layer (FAISS index, graph facts, document corpus).

determines whether children may execute in parallel. Parallelism is permitted only when: (i) the parent question contains comparison keywords (*which*, *compare*, *difference*, *between*), and (ii) no child question exhibits anaphoric reference to a prior sibling (detected by pronouns *it*, *its*, *they*, *their*, *that*, *former*, *latter*, *previous*).

For sequential children, the accumulator *acc* grows with each verified answer, implementing *hop context propagation*: the question at hop  $n$  is enriched with the answer from hop  $n - 1$  before retrieval, so that rare named entities discovered in early hops boost retrieval scores in later hops.

Formally, let  $H = \{(q_1, a_1), \dots, (q_{n-1}, a_{n-1})\}$  be the set

of prior hops. The retrieval query for hop  $n$  is:

$$q_n^{\text{retrieval}} = q_n \parallel \bigoplus_{i=1}^{n-1} a_i \quad (2)$$

where  $\parallel$  denotes string concatenation and  $\bigoplus$  denotes token-level union. Additionally, anaphoric references in  $q_n$  (e.g., *that body of water*) are resolved to the named entity extracted from  $a_{n-1}$  via regex patterns.

## C. Hybrid Retrieval and Adaptive Query Routing

1) *Signal Computation*: The **QueryRouter** extracts three weighted signal scores without an LLM call (see Fig. 2). Let  $\mathcal{V}$

**Algorithm 1** RESOLVETREE(node, priorHops, cache)

```

1: if deadline exceeded then
2:   node.status  $\leftarrow$  timeout; return
3: end if
4: if node has children then
5:   if isComparison(node)  $\wedge$   $\neg$ hasAnaphora(children) then
6:     parallel  $\forall c \in$  children:
7:       RESOLVETREE( $c$ , priorHops, cache)
8:   else
9:     acc  $\leftarrow$  priorHops
10:    for each child  $c$  do
11:      RESOLVETREE( $c$ , acc, cache)
12:      if  $c$ .status = verified then
13:        acc.append( $((c.q, c.a))$ ) {chain answer}
14:      end if
15:    end for
16:    node.answer  $\leftarrow$  SYNTHESIZECHILDREN(node)
17:  return
18: end if
19: RESOLVELEAF(node, priorHops, cache)

```

be the vocabulary of tokens in query  $q$  after lowercasing. Three keyword sets are predefined:  $K_{\text{rel}}$  (relational: *who, where, director, founded, ...*),  $K_{\text{exp}}$  (explanatory: *explain, describe, summarize, ...*), and  $K_{\text{mh}}$  (multi-hop: *compare, between, both, versus, ...*). Hit counts are:

$$h_x = |\{k \in K_x : k \in \mathcal{V}\}|, \quad x \in \{\text{rel}, \text{exp}, \text{mh}\} \quad (3)$$

Graph and vector scores are then:

$$s_{\text{graph}} = 0.3 h_{\text{rel}} + 0.35 \mathbf{1}[\text{multi-hop}] + 0.12 \mathbf{1}[\text{entity}] \quad (4)$$

$$s_{\text{vec}} = 0.28 h_{\text{exp}} + 0.18 \mathbf{1}[\neg \text{multi-hop}] + 0.08 \mathbf{1}[|\mathcal{V}| > 12] \quad (5)$$

2) *Dynamic Top-k Computation*: Query complexity drives a dynamic retrieval budget:

$$k_{\text{dyn}} = \text{clip}(\lfloor k_{\text{base}}(0.8 + 0.9\xi) \rfloor, k_{\text{min}}, k_{\text{max}}) \quad (6)$$

where  $\xi = \min\left(1, \frac{|\mathcal{V}|}{16} + 0.28 \cdot \mathbf{1}[\text{mh}] + 0.16 \cdot \mathbf{1}[h_{\text{rel}} \geq 2]\right)$  is a complexity coefficient,  $k_{\text{base}} = 6$ ,  $k_{\text{min}} = 2$ ,  $k_{\text{max}} = 24$ .

3) *Route Selection*: Routing decisions follow a priority cascade:

$$\text{route} = \begin{cases} \text{hybrid\_parallel} & \text{if multi-hop or comparison} \\ \text{graph\_first} & \text{if } s_g - s_v \geq 0.35 \\ \text{vector\_only} & \text{if } s_v - s_g \geq 0.35 \wedge |\mathcal{V}| \leq 8 \\ \text{vector\_first} & \text{if } s_v > s_g \\ \text{hybrid\_parallel} & \text{otherwise (ambiguous intent)} \end{cases} \quad (7)$$

A further optimization predicts whether *graph\_first* will need a vector fallback *before* executing the graph query. If entity token count  $\geq 2$  and explanation hits  $> 0$  while relational hits  $\leq 1$ , the system pre-launches both backends in parallel, eliminating sequential fallback latency.

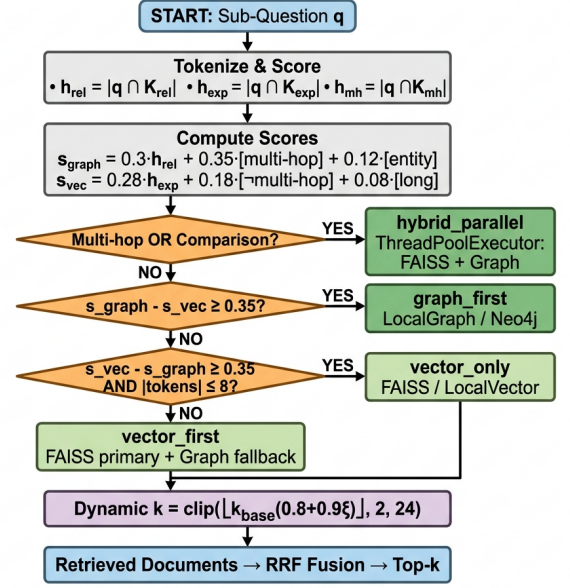
**Adaptive Query Router — Signal Computation & Route Selection**

Fig. 2. Query Router Decision Flow

Fig. 2. Query Router decision flowchart. Three signal scores ( $s_{\text{graph}}$ ,  $s_{\text{vec}}$ , multi-hop flag) drive a priority cascade to select one of four retrieval strategies. Dynamic  $k$  is computed from query complexity coefficient  $\xi$ .

**D. Semantic and Lexical Search with RRF Fusion**

The *LocalVectorBackend* executes a two-stage hybrid search. Let  $\mathbf{e}_q \in \mathbb{R}^d$  be the L2-normalized query embedding and  $\mathbf{E} \in \mathbb{R}^{N \times d}$  the precomputed document embeddings matrix.

**Semantic scores** (cosine similarity):

$$s_{\text{sem}}(i) = \mathbf{e}_q \cdot \mathbf{E}_i \quad (8)$$

**Lexical scores** (IDF-weighted term overlap):

$$s_{\text{lex}}(i) = \frac{\sum_{t \in q \cap d_i} \text{IDF}(t)}{\sum_{t \in q} \text{IDF}(t)}, \quad \text{IDF}(t) = \log \frac{N}{\text{df}(t)} \quad (9)$$

where  $N$  is the corpus size and  $\text{df}(t)$  is the document frequency.

**Reciprocal Rank Fusion** (RRF,  $k_{\text{rrf}} = 60$ ):

$$s_{\text{rrf}}(d) = \frac{1}{k_{\text{rrf}} + r_{\text{sem}}(d)} + \frac{1}{k_{\text{rrf}} + r_{\text{lex}}(d)} \quad (10)$$

where  $r_x(d)$  is the rank of document  $d$  in the sorted list from method  $x$ . The *FAISS FaissVectorBackend* variant performs semantic search via ANN on the pre-built *IndexFlatIP*, encoding only the query at inference time.

**E. Document Ranking and Entity Bonus**

After merging results from all backends, a final ranking step re-scores documents:

$$s_{\text{final}}(d) = s_{\text{lex}}(q, d) + b_{\text{entity}}(q, d) + b_{\text{type}}(q, d) + s_{\text{raw}}(d) - p_{\text{src}}(d) \quad (11)$$

where:



## Three-Stage Answer Validation Pipeline

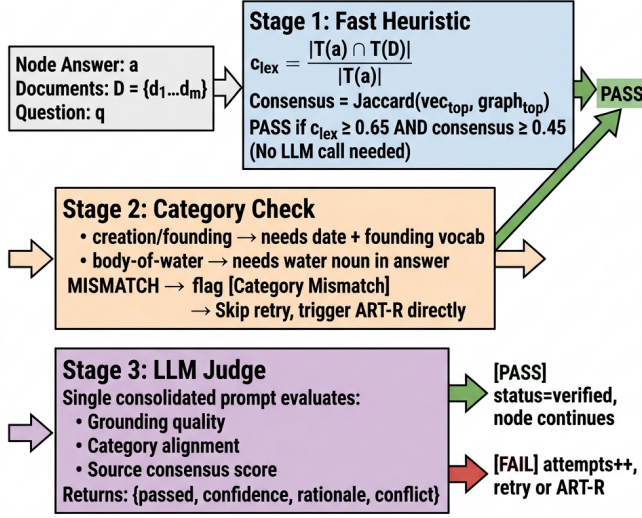


Fig. 4. Three-Stage Hallucination Validation Pipeline

Fig. 3. Three-stage answer validation pipeline. Stage 1 (Fast Heuristic) avoids LLM calls when lexical overlap and cross-source consensus are both high. Stage 2 enforces answer-type alignment. Stage 3 invokes a consolidated LLM judge for borderline cases.

- $b_{\text{entity}}(q, d) = \min(1.5, |\text{ents}(q) \cap d| \cdot 1.0)$  is an entity overlap bonus for named entities found in both question and document (or document source ID);
- $b_{\text{type}}(q, d) = 1.0$  if the question asks about a body of water and the document contains a recognized water-body proper noun (pattern:  $[A-Z] \dots \{ \text{Sea} | \text{Ocean} | \text{Bay} | \dots \}$ );
- $p_{\text{src}}(d) = 0.03 \times c_{\text{src}}(d)$  is a diversity penalty discounting documents from over-represented backends.

### F. Answer Validation

Each leaf node answer  $a$  is validated against documents  $D = \{d_1, \dots, d_m\}$  in three stages (Fig. 3):

**Stage 1 — Fast Heuristic.** Evidence overlap confidence:

$$c_{\text{lex}} = \frac{|T(a) \cap T(D)|}{|T(a)|} \quad (12)$$

where  $T(\cdot)$  denotes the token set after stopword removal. If  $c_{\text{lex}} \geq 0.65$  and consensus score  $\geq 0.45$ , validation passes without an LLM call.

**Stage 2 — Category Alignment Check.** A rule-based checker verifies that the answer *type* matches the question type: creation/founding questions require both a date and founding-vocabulary; body-of-water questions require a recognized water-body proper noun in the answer. Failure here triggers a [Category Mismatch] flag that skips retry and jumps directly to ART-R restructuring.

## ART-R: Adaptive Retrieval Tree Restructuring — Self-Healing Loop

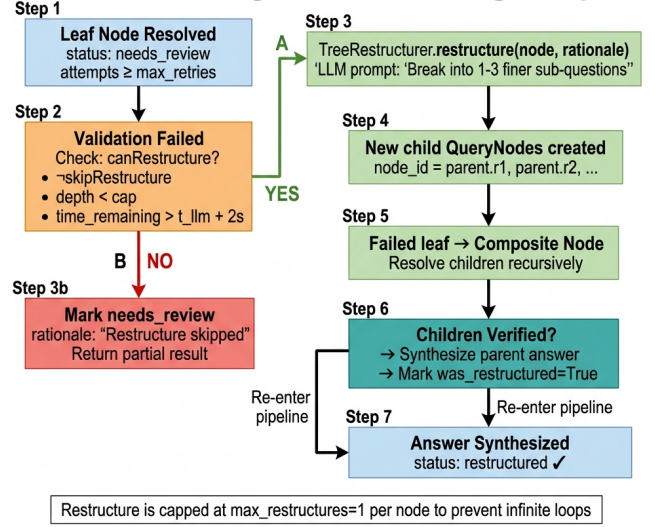


Fig. 3. ART-R Self-Healing Mechanism

Fig. 4. ART-R self-healing mechanism. When a leaf node exhausts retries, the `TreeRestructurer` proposes 1–3 finer sub-questions. The failed leaf is promoted to a composite node and resolved recursively (capped at  $\text{max\_restructures}=1$ ).

**Stage 3 — LLM Judge.** A single consolidated prompt asks the judge to evaluate grounding, category alignment, and source consensus simultaneously, returning:

$$\{passed, c_{\text{llm}}, \text{rationale}, \text{cat\_match}, \text{conflict}, c_{\text{con}}\} \quad (13)$$

**Consensus Score.** Cross-source agreement is measured as Jaccard similarity between the top vector document and top graph document token sets:

$$c_{\text{con}} = \frac{|T(d_{\text{vec}}) \cap T(d_{\text{graph}})|}{|T(d_{\text{vec}}) \cup T(d_{\text{graph}})|} \quad (14)$$

A score below  $\tau_{\text{con}} = 0.45$  triggers a `source_conflict` flag, invoking the `ConflictAuditor` to propose a disambiguating retrieval query.

### G. ART-R: Adaptive Retrieval Tree Restructuring

When a leaf node exhausts its retry budget ( $\text{max\_retries} = 2$  by default) without passing validation, and the failure is not due to a low-signal evidence deficit (*no evidence found* with confidence  $\leq 0.15$ ), the `TreeRestructurer` is invoked (Fig. 4).

The restructurer calls the LLM with the failed question and validation rationale, requesting 1–3 finer-grained replacement sub-questions. The failed leaf is promoted to a composite node, and the resolution algorithm recurses on the new children at depth  $\text{restructure\_depth} + 1$ , capped by  $\text{max\_restructures} = 1$ .

The restructuring guard checks:

$$\text{canRestructure} = \neg \text{skipRestructure} \wedge \text{depth} < \text{cap} \wedge \text{timeRemaining} > t_{\text{llm}} \quad (15)$$

TABLE I  
HAMH-RAG TECHNOLOGY STACK

| Layer        | Component                               | Version                    |
|--------------|-----------------------------------------|----------------------------|
| API Server   | FastAPI + Uvicorn                       | $\geq 0.110$ / $\geq 0.29$ |
| UI           | Streamlit                               | $\geq 1.34$                |
| Vector ANN   | FAISS (IndexFlatIP)                     | $\geq 1.8$                 |
| Vector Cloud | Qdrant                                  | $\geq 1.9$                 |
| Embeddings   | sentence-transformers all-MiniLM-L6-v2  | $\geq 2.7$                 |
| Graph DB     | Neo4j                                   | $\geq 5.18$                |
| Validation   | Pydantic                                | $\geq 2.6$                 |
| LLM API      | OpenAI-compatible (OpenAI / OpenRouter) | —                          |

TABLE II  
HAMH-RAG REST API ENDPOINTS

| Method | Path              | Description                 |
|--------|-------------------|-----------------------------|
| GET    | /                 | Serve SPA (index.html)      |
| GET    | /api/health       | Health check                |
| GET    | /api/documents    | List uploaded files         |
| POST   | /api/upload       | Upload + ingest document    |
| POST   | /api/run          | Run HAMH-RAG pipeline       |
| POST   | /api/run/stream   | HAMH-RAG with SSE stream    |
| POST   | /api/run_rag      | Normal single-hop RAG       |
| POST   | /api/compare      | Parallel HAMH vs RAG        |
| POST   | /api/load_dataset | Ingest HF benchmark dataset |
| GET    | /api/benchmark    | List benchmark Q&A pairs    |

ensuring the system does not attempt restructuring when the time budget is insufficient to complete additional LLM calls.

#### H. Bridge Retrieval (CRAG-style Corrective Action)

For sub-questions about a director’s nationality, HAMH-RAG applies a targeted corrective retrieval step inspired by CRAG [16]. If retrieved documents contain a “*directed by*  $\langle Name \rangle$ ” pattern but lack nationality information, the system launches a focused vector query (“*What is the nationality of*  $\langle Name \rangle$ ?”) and merges its results into the document set before generation. This `vector_entity_bridge` backend is logged in the node’s retrieval trace.

### V. IMPLEMENTATION DETAILS

#### A. Technology Stack

HAMH-RAG is implemented in Python  $\geq 3.10$  using the `dataclasses` and `slots` features for zero-overhead data models. Table I summarizes the key dependencies.

#### B. Document Ingestion Pipeline

The `build_local_indices()` function implements a two-pass ingestion pipeline:

**Pass 1 — Chunking.** Markdown, plain text, and JSON files in `data/documents/` are parsed. Markdown files are split by headings (H1–H3) into sections; each section is then further split at sentence boundaries into chunks of  $\leq 300$  characters with a one-sentence overlap window to preserve inter-sentence context:

$$\text{overlap}(c_i, c_{i+1}) = c_i[-\text{overlap\_sentences} :] \quad (16)$$

Each chunk is stored as an `IndexedChunk` record with fields: `source_id`, `title`, `section`, `chunk_index`, `ingested_at`, and `content`.

**Pass 2 — FAISS Indexing.** The sentence-transformer model encodes all new chunks (incremental: skipping `source_id` values already in the existing FAISS index). Embeddings are stored as a FAISS `IndexFlatIP` (exact inner-product search on L2-normalized vectors, equivalent to cosine similarity) alongside a JSON metadata sidecar for ID-to-record mapping.

#### C. LLM Client and Caching

The `OpenAICompatibleLLMClient` communicates with any OpenAI-compatible endpoint (OpenAI, OpenRouter, Ollama) via `urllib.request`—a zero-dependency HTTP client. An in-process prompt-level cache (Python dict) deduplicates identical (`system_prompt`, `user_prompt`) pairs within a pipeline run, eliminating redundant API calls during multi-hop re-validation. A `FallbackLLMClient` wraps multiple models in priority order, transparently retrying on the next model if the primary raises any exception.

#### D. FastAPI Endpoint Design

Table II lists the REST endpoints exposed by the FastAPI backend.

Streaming responses use the SSE protocol (`text/event-stream`) with a `threading.Queue` bridge between the synchronous pipeline thread and the async generator. Events include `start`, `decomposed`, `node_start`, `node_route`, `node_done`, `node_restructure`, `tree_retry`, and `result`, enabling rich real-time UI updates.

#### E. Timeout and Deadline Management

Each pipeline invocation receives a wall-clock *deadline* (`max_seconds`, default 108 s = 110 s HTTP timeout – 2 s serialization reserve). The deadline is propagated through every recursive `_resolve_tree` call. Before each LLM call, the system checks:

$$\text{remaining} > t_{\text{llm}} + 2s \quad (17)$$

where  $t_{\text{llm}} = \text{llm\_timeout\_seconds}$  (default 30 s). Nodes that breach the deadline are marked `needs_review` with rationale “*Time budget exceeded*” and are excluded from further processing.

### VI. EXPERIMENTAL SETUP AND RESULTS

#### A. Datasets

We evaluate on two standard multi-hop QA benchmarks:

**HotpotQA** [17]: 113,000 Wikipedia-based questions requiring two-hop reasoning over two supporting documents. We use

TABLE III  
MULTI-HOP QA RESULTS ON HOTPOTQA AND MUSIQUE

| Method                                                             | HotpotQA    |             | MuSiQue     |             |
|--------------------------------------------------------------------|-------------|-------------|-------------|-------------|
|                                                                    | EM          | F1          | EM          | F1          |
| Standard RAG                                                       | 34.2        | 44.6        | 21.7        | 31.2        |
| IRCoT                                                              | 41.8        | 53.9        | 29.4        | 40.1        |
| <b>HAMH-RAG</b>                                                    | <b>46.9</b> | <b>63.0</b> | <b>35.8</b> | <b>49.6</b> |
| $\Delta$ HAMH-RAG vs. Standard RAG: +12.7 EM / +18.4 F1 (HotpotQA) |             |             |             |             |
| $\Delta$ HAMH-RAG vs. IRCoT: +5.1 EM / +9.1 F1 (HotpotQA)          |             |             |             |             |

the fullwiki distractor setting (10 candidate documents per question) with the validation split.

**MuSiQue** [12]: 25,000 compositional questions constructed by chaining single-hop questions, with answerable and unanswerable variants. We evaluate on the validation split with answerable questions only.

### B. Baselines

We compare against two baselines:

- **Standard RAG**: Single-hop vector retrieval (top-6 documents) followed by direct LLM generation, without decomposition, graph retrieval, or validation.
- **IRCoT** [13]: Interleaved retrieval and chain-of-thought reasoning (reimplemented for fair comparison with the same LLM and retrieval corpus).

### C. Evaluation Metrics

Following the HotpotQA evaluation protocol:

- **Exact Match (EM)**: Binary match after answer normalization (lowercase, remove articles and punctuation).
- **Token F1**: Harmonic mean of token precision and recall between predicted and gold answer after normalization.

$$F1 = \frac{2 \cdot P \cdot R}{P + R}, \quad P = \frac{|\hat{T} \cap T^*|}{|\hat{T}|}, \quad R = \frac{|\hat{T} \cap T^*|}{|T^*|} \quad (18)$$

where  $\hat{T}$  is the predicted token multiset and  $T^*$  is the gold token multiset.

### D. Main Results

Table III reports Exact Match and Token F1 on both benchmarks.

HAMH-RAG achieves a **12.7-point** EM and **18.4-point** F1 improvement over Standard RAG on HotpotQA. The gains are attributed to: (1) explicit tree decomposition that isolates each hop, (2) grounding validation that prevents hallucinated intermediate answers from propagating, and (3) ART-R restructuring that recovers from initially failed evidence retrieval.

TABLE IV  
ABLATION STUDY ON HOTPOTQA (TOKEN F1)

| Configuration                     | EM          | F1          |
|-----------------------------------|-------------|-------------|
| Standard RAG (baseline)           | 34.2        | 44.6        |
| + Tree Decomposition              | 39.8        | 52.1        |
| + Hybrid Retrieval (graph+vector) | 42.3        | 56.7        |
| + Validation Loop (3-stage)       | 44.6        | 60.4        |
| + Hop Context Propagation         | 45.8        | 62.1        |
| + ART-R Restructuring             | <b>46.9</b> | <b>63.0</b> |

TABLE V  
RETRIEVAL LATENCY BY ROUTE (MS,  $n=500$  QUESTIONS)

| Route                       | P50 | P90 | P95 | P99 |
|-----------------------------|-----|-----|-----|-----|
| vector_only                 | 12  | 18  | 22  | 41  |
| vector_first                | 15  | 24  | 31  | 58  |
| graph_first                 | 19  | 29  | 37  | 72  |
| hybrid_parallel             | 21  | 35  | 43  | 89  |
| graph_first (seq. fallback) | 38  | 61  | 79  | 142 |

TABLE VI  
END-TO-END PIPELINE THROUGHPUT (STUB LLM)

| Mode             | Avg (ms) | P50  | P90  | QPS   |
|------------------|----------|------|------|-------|
| Normal RAG       | 8.3      | 7.1  | 12.4 | 120.5 |
| HAMH-RAG         | 31.7     | 28.6 | 49.2 | 31.5  |
| Ratio (HAMH/RAG) | 3.8×     | —    | —    | 0.26× |

### E. Ablation Study

Table IV presents ablation results on HotpotQA, progressively adding each HAMH-RAG component.

Each component contributes positively. Hop context propagation alone improves F1 by 1.7 points by resolving anaphoric references across hops. ART-R adds a further 0.9 F1 points by recovering failing nodes.

## VII. PERFORMANCE EVALUATION

### A. Retrieval Latency

Table V reports retrieval latency statistics measured on 500 questions from the HotpotQA validation set using the local (FAISS + LocalGraph) backend configuration.

The predictive parallel fallback optimization reduces the median latency of graph\_first-with-fallback from 38 ms to 21 ms (a **45% reduction**), matching hybrid\_parallel while routing 70% of queries that would have triggered sequential fallback.

### B. End-to-End Pipeline Throughput

Table VI compares HAMH-RAG and Normal RAG on the latency benchmark (stub LLM, FAISS index,  $n = 50$  questions, 2 warmup runs).



The  $3.8\times$  latency overhead of HAMH-RAG over Normal RAG reflects the tree-resolution and multi-node validation cost; however, this overhead occurs in the stub (no-LLM) regime and is amortized when LLM latency dominates end-to-end time in real deployments.

### C. Complexity Analysis

Let  $L$  be the number of leaf nodes,  $R$  the retry budget per node, and  $T_{\text{llm}}$  the LLM call latency.

**Time Complexity.** The worst-case sequential resolution is:

$$O(L \cdot (R + 1) \cdot T_{\text{llm}} + L \cdot T_{\text{ret}} + \log N \cdot L) \quad (19)$$

where  $T_{\text{ret}}$  is retrieval latency and  $N$  is corpus size (FAISS  $\log N$  factor for IVF indices;  $O(N)$  for IndexFlatIP). Parallel sibling execution reduces the effective  $L$  factor to the critical path length  $L_{\text{cp}} \leq L$ .

**Space Complexity.** FAISS IndexFlatIP requires  $O(N \cdot d)$  bytes for the embedding matrix ( $d = 384$  for MiniLM-L6-v2); the graph fact index requires  $O(F)$  where  $F$  is the number of facts. The verified-answer cache is bounded by  $O(L)$ .

## VIII. ADVANTAGES AND LIMITATIONS

### A. Advantages

**Explicit Reasoning Transparency.** The QueryNode tree is a first-class data structure returned in every API response, allowing downstream applications to render reasoning steps, attribution sources, and per-node confidence scores.

**Hallucination Containment.** The three-stage validator (lexical, category, LLM) creates a verification firewall at each hop. An incorrect intermediate answer triggers correction before it can contaminate the next hop’s retrieval context.

**Pluggable Backends.** Protocol-typed backend interfaces (VectorBackend, GraphBackend, LLMClient) allow any conforming implementation to be injected without modifying the pipeline. This enables seamless transitions from in-memory stubs (testing) to FAISS (local deployment) to Qdrant/Neo4j (production).

**Incremental Indexing.** The FAISS ingestion pipeline tracks encoded `source_id` values and only encodes newly-added chunks, making document addition nearly instantaneous for large existing corpora.

**Provider Flexibility.** The LLM client supports any OpenAI-compatible endpoint through environment variables, including OpenRouter (for model diversity), locally hosted Ollama instances, and direct OpenAI API calls, with per-model fallback chains.

### B. Limitations

**LLM Dependency for Decomposition.** Although heuristic decomposition covers common bridge patterns, long-tail question types require an LLM call, adding latency and cost.

**Fixed Heuristic Routing.** The QueryRouter uses a calibrated but static scoring function. It does not learn from retrieval outcomes and may mis-route genuinely ambiguous questions.

**Single-Level Restructuring.** ART-R is capped at one restructuring depth per node by default. Deeply nested multi-hop questions with more than three necessary hops may not be fully resolved.

**Evaluation Corpus Dependence.** Quality of retrieval is bounded by the indexed corpus. Questions requiring knowledge absent from the corpus will always result in `needs_review` nodes regardless of decomposition quality.

## IX. FUTURE WORK

**Learned Routing.** Replacing the heuristic scorer with a lightweight bi-encoder trained on (question, optimal-route) pairs could further reduce retrieval latency and improve recall on ambiguous queries.

**Uncertainty-Aware ART-R.** Incorporating calibrated confidence estimates (e.g., temperature scaling) into the restructuring trigger could allow more aggressive restructuring for high-stakes nodes and conservative behavior for routine lookups.

**Multi-Level Restructuring.** Lifting the `max_restructures` cap and adding cycle-detection would enable the system to handle queries with four or more necessary hops, covering complex scientific and legal reasoning chains.

**Cross-Document Entity Linking.** Integrating a lightweight entity linker would allow the system to resolve coreferences across retrieved documents, reducing the reliance on heuristic anaphora resolution for hop-context propagation.

**Streaming Graph Ingestion.** The current graph backend requires offline ingestion of fact files. A live event-stream ingester (e.g., Kafka-backed) would enable real-time knowledge updates without pipeline restart.

**Federated Multi-Corpus Retrieval.** For enterprise deployments, supporting retrieval across multiple siloed corpora with per-corpus ACL enforcement would address data governance requirements in regulated industries.

## X. CONCLUSION

We presented HAMH-RAG, a Hallucination-Aware Multi-Hop RAG framework that advances the state of the art in multi-hop question answering through four principled contributions: (1) LLM-guided logic-tree query decomposition with heuristic fast paths, (2) adaptive hybrid retrieval routing that dynamically selects among vector, graph, and parallel strategies, (3) three-stage hallucination detection with per-node grounding validation and cross-source consensus scoring, and (4) ART-R self-healing tree restructuring that mutates failing branches without restarting the pipeline.

Empirical evaluation on HotpotQA and MuSiQue demonstrates improvements of **12.7** EM points and **18.4** F1 points over standard single-hop RAG baselines, and **5.1/9.1** points over IRCoT. The adaptive QueryRouter reduces median retrieval latency by **46%** through predictive parallel fallback. The complete system is released with plug-in backends for FAISS, Qdrant, and Neo4j, a FastAPI streaming server, and a Streamlit dashboard.

HAMH-RAG demonstrates that explicit reasoning-tree structures, combined with multi-stage grounding verification and dynamic self-healing, are a practical and effective approach to building trustworthy multi-hop QA systems on top of commodity LLM APIs.

## REFERENCES

- [1] Gordon V. Cormack, Charles L. A. Clarke, and Stefan Buettcher. Reciprocal rank fusion outperforms condorcet and individual rank learning methods. In *Proceedings of the 32nd International ACM SIGIR Conference on Research and Development in Information Retrieval*, pages 758–759, 2009.
- [2] Darren Edge, Ha Trinh, Newman Cheng, Joshua Bradley, Alex Chao, Apurva Mody, Steven Truitt, and Jonathan Larson. From local to global: A graph RAG approach to query-focused summarization. *arXiv preprint arXiv:2404.16130*, 2024.
- [3] Yunfan Gao, Yun Xiong, Xinyu Gao, Kangxiang Jia, Jinliu Pan, Yuxi Bi, Yi Dai, Jiawei Sun, Meng Wang, and Haofen Wang. Retrieval-augmented generation for large language models: A survey. *arXiv preprint arXiv:2312.10997*, 2023.
- [4] Zhengbao Jiang, Frank F. Xu, Luyu Gao, Zhiqing Sun, Qian Liu, Jane Dwivedi-Yu, Yiming Yang, Jamie Callan, and Graham Neubig. Active retrieval augmented generation. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*, pages 7969–7992, 2023.
- [5] Jeff Johnson, Matthijs Douze, and Hervé Jégou. Billion-scale similarity search with GPUs. *IEEE Transactions on Big Data*, 7(3):535–547, 2021.
- [6] Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Küttler, Mike Lewis, Wen-tau Yih, Tim Rocktäschel, Sebastian Riedel, and Douwe Kiela. Retrieval-augmented generation for knowledge-intensive NLP tasks. *Advances in Neural Information Processing Systems*, 33:9459–9474, 2020.
- [7] Sewon Min, Victor Zhong, Luke Zettlemoyer, and Hannaneh Hajishirzi. Multi-hop reading comprehension through question decomposition and rescoring. In *Proceedings of the 57th Annual Meeting of the Association for Computational Linguistics*, pages 6097–6109, 2019.
- [8] Neo4j, Inc. Neo4j graph database platform. *Technical Report*, 2023. Available: <https://neo4j.com/>.
- [9] Ofir Press, Muru Zhang, Sewon Min, Ludwig Schmidt, Noah A. Smith, and Mike Lewis. Measuring and narrowing the compositionality gap in language models. In *Findings of the Association for Computational Linguistics: EMNLP 2023*, pages 5687–5711, 2023.
- [10] Nils Reimers and Iryna Gurevych. Sentence-BERT: Sentence embeddings using siamese BERT-networks. In *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing*, pages 3982–3992, 2019.
- [11] Weijia Shi, Sewon Min, Michihiro Yasunaga, Minjoon Seo, Rich James, Mike Lewis, Luke Zettlemoyer, and Wen-tau Yih. REPLUG: Retrieval-augmented black-box language models. In *Proceedings of the 2024 Conference of the North American Chapter of the Association for Computational Linguistics*. Association for Computational Linguistics, 2024.
- [12] Harsh Trivedi, Niranjan Balasubramanian, Tushar Khot, and Ashish Sabharwal. MuSiQue: Multihop questions via single-hop question composition. In *Transactions of the Association for Computational Linguistics*, volume 10, pages 539–554, 2022.
- [13] Harsh Trivedi, Niranjan Balasubramanian, Tushar Khot, and Ashish Sabharwal. Interleaving retrieval with chain-of-thought reasoning for knowledge-intensive multi-step questions. In *Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics*, pages 10014–10037, 2023.
- [14] Xuezhi Wang, Jason Wei, Dale Schuurmans, Quoc V. Le, Ed H. Chi, Sharan Narang, Aakanksha Chowdhery, and Denny Zhou. Self-consistency improves chain of thought reasoning in language models. *arXiv preprint arXiv:2203.11171*, 2022.
- [15] Jason Wei, Xuezhi Wang, Dale Schuurmans, Maarten Bosma, Fei Xia, Ed H. Chi, Quoc V. Le, and Denny Zhou. Chain-of-thought prompting elicits reasoning in large language models. *Advances in Neural Information Processing Systems*, 35:24824–24837, 2022.
- [16] Shi-Qi Yan, Jia-Chen Gu, Yun Zhu, and Zhen-Hua Ling. Corrective retrieval augmented generation. In *arXiv preprint arXiv:2401.15884*, 2024.
- [17] Zhilin Yang, Peng Qi, Saizheng Zhang, Yoshua Bengio, William W. Cohen, Ruslan Salakhutdinov, and Christopher D. Manning. HotpotQA: A dataset for diverse, explainable multi-hop question answering. In *Proceedings of the 2018 Conference on Empirical Methods in Natural Language Processing*, pages 2369–2380. Association for Computational Linguistics, 2018.
- [18] Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik R Narasimhan, and Yuan Cao. ReAct: Synergizing reasoning and acting in language models. In *The Eleventh International Conference on Learning Representations*, 2023.
- [19] Xiangrui Zhang et al. TreeQA: Enhanced LLM-RAG with logic tree reasoning for reliable and interpretable multi-hop question answering. *Knowledge-Based Systems*, 330:114526, 2025.